

SMART GROCERY APP

Project Proposal & Technical Documentation

A Native Android Application

Jetpack Compose • Kotlin Ktor • MongoDB Atlas

1. Project Objective

Develop a modern, full-featured Android grocery shopping application that allows users to browse products by category, search for items instantly, manage a shopping cart, and place orders through a clean, intuitive, and responsive interface.

The application follows a fully native architecture: the frontend is built entirely with Jetpack Compose, while the backend is powered by a Kotlin Ktor REST API connected to a cloud-hosted MongoDB Atlas database — meeting the requirement to implement both the client and server layers in Kotlin.

2. Technology Stack

Frontend

- Kotlin
- Jetpack Compose — declarative UI toolkit
- Material Design 3
- Navigation Compose
- ViewModel (MVVM architecture)
- StateFlow — reactive state management

Backend

- Kotlin
- Ktor Framework — asynchronous web server
- REST API (JSON over HTTP)
- Kotlin Serialization
- Coroutines — asynchronous, non-blocking I/O

Database

- MongoDB Atlas — cloud-hosted NoSQL database

Development Tools

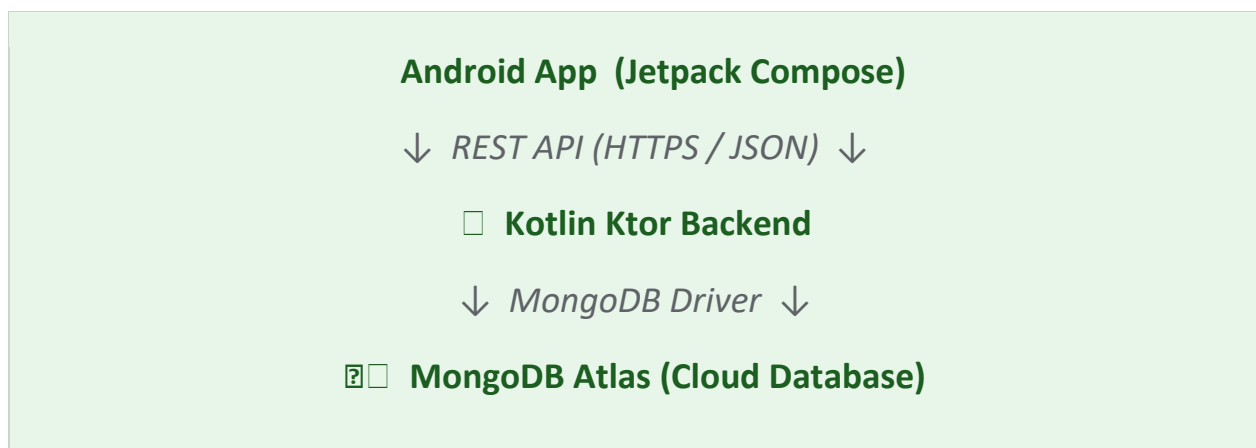
- Android Studio
- Git & GitHub — version control
- Postman — API testing
- MongoDB Compass — database management (optional)

Final Tech Stack Summary

Component	Technology
Frontend	Kotlin + Jetpack Compose
Backend	Kotlin + Ktor
Database	MongoDB Atlas
API Style	REST API
IDE	Android Studio
Version Control	Git & GitHub
API Testing	Postman
DB Management	MongoDB Compass (Optional)

3. System Architecture

The application follows a clean three-tier architecture, ensuring clear separation of concerns between the presentation, business logic, and data layers.



4. Grocery Categories

Products are organized into clearly defined categories to make browsing fast and intuitive:

Icon	Category
	Vegetables & Fruits
	Dairy Products
	Basic Medicines
	Bakery Items
	Rice & Grains
	Beverages
	Snacks
	Household Essentials

5. Core Features

User Authentication

- User Registration
- User Login
- Logout

Product Features

- Product Listing
- Product Details View

- Search Products
- Filter by Category
- Product Images
- Product Pricing
- Stock Availability

Shopping Cart

- Add to Cart
- Remove from Cart
- Update Quantity
- Automatic Total Price Calculation

Order Management

- Place Order
- Order History
- Order Details
- Order Status Tracking

User Profile

- Name
- Email
- Phone Number
- Delivery Address

6. Backend API Modules

Module	Responsibility
Authentication API	Handles registration, login, logout, and session/token management
Category API	Serves the list of grocery categories
Product API	Handles product listing, details, search, and filtering
Cart API	Manages add / remove / update operations on the user's cart
Order API	Handles order placement, history, and status updates
User API	Manages user profile data

7. Data Models (MongoDB Collections)

The database uses five core collections. Each document uses MongoDB's default ObjectId (`_id`) as its primary key unless otherwise noted.

Users (*MongoDB Collection*)

Field	Type	Description
<code>_id</code>	ObjectId	Unique user identifier
<code>name</code>	String	Full name of the user
<code>email</code>	String	Unique email address (used for login)

Field	Type	Description
passwordHash	String	Encrypted/hashed password
phoneNumber	String	Contact number
address	String	Default delivery address
createdAt	DateTime	Account creation timestamp

📁 **Categories** (MongoDB Collection)

Field	Type	Description
_id	ObjectId	Unique category identifier
name	String	Category name, e.g. "Vegetables & Fruits"
icon	String	Emoji or icon reference
imageUrl	String	Category banner/thumbnaill image
displayOrder	Int	Order in which category appears in the UI

📁 **Products** (MongoDB Collection)

Field	Type	Description
_id	ObjectId	Unique product identifier
name	String	Product name
description	String	Product description

Field	Type	Description
categoryId	ObjectId	Reference to a Categories document
price	Double	Unit price
unit	String	Unit of measure, e.g. "kg", "pack", "litre"
imageUrl	String	Product image
stockQuantity	Int	Units currently available
isAvailable	Boolean	Availability flag
createdAt	DateTime	Record creation timestamp

Cart (MongoDB Collection)

Field	Type	Description
_id	ObjectId	Unique cart identifier
userId	ObjectId	Reference to the owning user
items	Array<CartItem>	List of cart items (see below)
updatedAt	DateTime	Last modification timestamp

CartItem (embedded sub-document):

Field	Type	Description
productId	ObjectId	Reference to a Products document
quantity	Int	Number of units selected

Field	Type	Description
priceAtAdd	Double	Product price captured at the time it was added

Orders *(MongoDB Collection)*

Field	Type	Description
_id	ObjectId	Unique order identifier
userId	ObjectId	Reference to the ordering user
items	Array<OrderItem>	Ordered products, quantities, and prices
totalAmount	Double	Total order value
deliveryAddress	String	Address for this order
status	String	"Pending" "Confirmed" "Shipped" "Delivered" "Cancelled"
placedAt	DateTime	Order placement timestamp

8. Project Workflow

- User creates an account or logs in.
- User browses grocery categories.
- User views available products within a category.
- User adds selected products to the shopping cart.
- User places an order.
- Order information is persisted in MongoDB Atlas.
- User can view previous orders and their status.

9. Future Enhancements

The following features are proposed as optional extensions beyond the initial scope:

- Favorites / Wishlist
- Push Notifications
- QR Code Scanner
- Google Maps integration for delivery address selection
- Online Payments (Stripe / JazzCash / EasyPaisa)
- Admin Dashboard
- Product Reviews & Ratings

10. Conclusion

The Smart Grocery App combines a modern, fully native Android frontend built with Jetpack Compose and a lightweight, high-performance Kotlin Ktor backend backed by MongoDB Atlas. This stack keeps the entire codebase in Kotlin end-to-end, satisfies the academic requirement for a Kotlin-based backend, and results in a scalable, maintainable, and professional-grade grocery shopping application.